

SOFTWARE ENGINEERING PRACTICE TUTORIAL

Tutorial 02

Programming Decisions, Loops, and Functions

Build and test a simple Arduino status classifier in Tinkercad simulation

ENGINEERING FOCUS This week, you will turn simple rules into code, test the important boundary values, and fix a logic error. The Arduino is our practice environment; the same thinking applies to larger software systems.

Course information	Details
Course	COMP6100001 — Software Engineering
Semester	Odd Semester 2026–2027
Tutorial	02 — Week 2A core practice; Week 2B completion and troubleshooting window
Mode	Individual work with limited peer discussion and declared AI assistance
Prepared by	Ruixin Huang
Official contact	ruixin.huang@binus.ac.id

STUDENT IDENTIFICATION

Name	Student ID	Class	Date started
Michael Arianno Chandrarieta	2802499711	B5BC	16/06/2026

HOW TO USE THIS TUTORIAL Complete the prediction before running the code. Record what you actually observe, even when it is different from your prediction.

1. Purpose and learning outcomes

In Week 1, you practised the basic development flow: understand the request, implement it, and verify the result. In Week 2, you will use that same flow with more programming logic.

1.1 By the end of this tutorial, you should be able to

- use constants and variables to store values clearly;
- use comparison and Boolean operators in conditions;
- write an if / else if / else decision;
- use a for loop to repeat a test;

- place repeated logic inside a function;
- test values below, at, and above a decision boundary;
- find and fix a logical error that still compiles.

1.2 Tutorial workflow

Part	Main question	What you will produce
A. Understand	What should the program do?	Personal settings and decision rules
B. Predict	What output should each value produce?	Prediction table
C. Build	How do we express the rules in code?	Working personalized program
D. Test	Does the code work at the boundaries?	Test results and evidence
E. Debug	Why does the defective code fail?	Defect record and corrected code
F. Improve	Can the code be clearer without changing behavior?	Small refactoring and explanation

1.3 Suggested time

Activity	Suggested time
Preparation and prediction	10–15 minutes
Build the program	20–25 minutes
Test the boundaries	15–20 minutes
Debug and improve	15–20 minutes
Complete evidence and reflection	10–15 minutes

IMPORTANT Do not rush directly to the final code. The prediction, testing, and debugging records are part of the tutorial—not optional paperwork.

2. Preparation and environment check

2.1 What you need

- ☐ A laptop with a current web browser.
- ☐ A Tinkercad account and access to Circuits.

❑ The Week 2A slides and this tutorial.

❑ A folder named <StudentID>_COMP6100001_T02.

❑ A text editor for saving your final code.

Tinkercad: <https://www.tinkercad.com/>

Arduino language reference: <https://docs.arduino.cc/language-reference/>

2.2 Evidence folder

```
<StudentID>_COMP6100001_T02/
├── 01_final_source.ino or .txt
├── 02_evidence/
│   ├── checkpoint_1_working.png
│   ├── checkpoint_2_boundary_tests.png
│   └── checkpoint_3_bug_fixed.png
└── <StudentID>_COMP6100001_T02_<Name>.pdf
```

2.3 Environment check

Step	What to do
1	Open Tinkercad and choose Circuits.
2	Create a circuit named T02_<StudentID>_<Name>.
3	Add one Arduino Uno R3. You do not need an LED or sensor for this activity.
4	Open Code, select Text mode, and find the Serial Monitor.
5	Confirm that Start Simulation and Stop Simulation are available.

Checkpoint 0	Pass condition	Your evidence
--------------	----------------	---------------

Ready to begin Circuit name contains your ID and name; Arduino, code editor, and Serial Monitor are visible. Filename: checkpoint_1_working.png

3. Practice A — Understand the task

SCENARIO You are building a small system monitor. A simulated load value from 0 to 100 must be classified as NORMAL, WARNING, or CRITICAL. The result is shown in the Serial Monitor.

3.1 Decision rules

When the load is...	Status	Action
below the warning threshold	NORMAL	CONTINUE
at or above warning, but below critical	WARNING	CHECK
at or above the critical threshold	CRITICAL	STOP

3.2 Get your personal values

Use the last digit of your student ID. This gives each student a slightly different program and set of expected results.

Last ID digit	Warning threshold (W)	Critical threshold (C)	Loop step
0–2	40	75	10
3–5	45	80	10
6–7	50	85	10
8–9	55	90	10

Last ID digit	Interval (ms)	Keyword	Identity token
1	2000	Ready	MAC711

3.3 Requirements

ID	The program shall...
R1	start Serial communication at 9600 baud and print your identity once.
R2	report NORMAL:CONTINUE when load < W.
R3	report WARNING:CHECK when load >= W and load < C.
R4	report CRITICAL:STOP when load >= C.
R5	use a for loop to test load values from 0 to 100.

R6	place the decision logic inside reportLoad(int load).
----	---

4. Practice B — Predict before running

First, think about the important values around W and C. Do not run the program until you complete this table.

Test	Input	Your predicted status	Why?
P1	W - 1	Normal	below 40, fails both $\geq$ checks
P2	W	Warning	load $\geq$ 40 true, load $\geq$ 75 false
P3	W + 1	Warning	same range as P2
P4	C - 1	Warning	still below 75
P5	C	Critical	load $\geq$ 75 true
P6	C + 1	Critical	same range as P5

4.1 Quick code questions

Why must C be greater than W?

If $C \leq W$, the WARNING range (at or above warning stage and below critical) becomes empty or inverted, so a value could never be WARNING, or WARNING/CRITICAL logic would contradict itself.

What does && mean in a condition?

logical AND. The whole condition is only true if both sides are true.

What is the difference between = and ==?

= assigns a value to a variable.

== compares two values and returns true/false.

Why should every load value match only one status?

So behavior is deterministic — if ranges overlapped or had gaps, the same input could report different results, or none at all, which is a hidden bug.

5. Practice C — Build the program

5.1 Complete the starter code

Replace the four TODO values with your own values. Read the comments as you work.

```
const int WARNING_THRESHOLD = 40;
const int CRITICAL_THRESHOLD = 75;
const int STEP = 10;
const char IDENTITY_TOKEN[] = "MAC711";

void reportLoad(int load) {
  Serial.print("LOAD:");
  Serial.print(load);
  if (load >= CRITICAL_THRESHOLD) {
    Serial.println(":CRITICAL:STOP");
  } else if (load >= WARNING_THRESHOLD) {
    Serial.println(":WARNING:CHECK");
  } else {
    Serial.println(":NORMAL:CONTINUE");
  }
}

void setup() {
  Serial.begin(9600);
  Serial.print("STUDENT:");
  Serial.println(IDENTITY_TOKEN);

  for (int load = 0; load <= 100; load += STEP) {
    reportLoad(load);
  }
}

void loop() {
  // Empty because this test runs once.
}
```

CHECK BEFORE RUNNING All TODO values must be replaced. Your identity token should contain your initials and the last three digits of your student ID.

5.2 Run and understand the program

Step	What to do
1	Enter the completed code in Tinkercad.
2	Before starting, choose one NORMAL, one WARNING, and one CRITICAL value and trace them through the conditions.
3	Start the simulation and open Serial Monitor.
4	Check that your identity appears once.
5	Check that the load increases using your assigned step and does not go above 100.
6	Save checkpoint_1_working.png showing the circuit name, readable code, and output.

5.3 Explain your code

Question	Your explanation
Why is the CRITICAL condition checked first?	Because <code>CRITICAL_THRESHOLD > WARNING_THRESHOLD</code>. If we checked <code>WARNING</code> first, every critical value would also satisfy <code>>= WARNING_THRESHOLD</code> and will never reach <code>CRITICAL</code>.
What does the for loop control?	It drives load from 0 to 100 in steps (<code>STEP</code>), calling <code>reportLoad(load)</code> each time.
Why is <code>reportLoad(int load)</code> useful?	The same classification logic is reused for both the sweep and the boundary tests. One place to fix bugs, no duplicated code.
What would happen if the for loop were placed in <code>loop()</code> ?	It would repeat forever, spamming the same output over and over instead of running once.
Why do we use named constants for <code>W</code> , <code>C</code> , and <code>STEP</code> ?	Constants mean you change the value in one place if requirements change.

SOFTWARE ENGINEERING LINK Clear names and small functions make code easier to read, test, and change. These habits matter even when the program is small.

6. Practice D — Test the boundary values

The for loop may skip W or C. For example, a step of 10 does not produce 45. Add these six calls temporarily in setup() after the identity line and comment out the for loop.

```
reportLoad(WARNING_THRESHOLD - 1);
reportLoad(WARNING_THRESHOLD);
reportLoad(WARNING_THRESHOLD + 1);
reportLoad(CRITICAL_THRESHOLD - 1);
reportLoad(CRITICAL_THRESHOLD);
reportLoad(CRITICAL_THRESHOLD + 1);
```

6.1 Test record

Test	Requirement	Input	Expected	Observed	Result
T1	R2	W - 1	NORMAL	LOAD:39:NORMAL:CONTINUE	Pass
T2	R3	W	WARNING	LOAD:40:WARNING:CHECK	Pass
T3	R3	W + 1	WARNING	LOAD:41:WARNING:CHECK	Pass
T4	R3	C - 1	WARNING	LOAD:74:WARNING:CHECK	Pass
T5	R4	C	CRITICAL	LOAD:75:CRITICAL:STOP	Pass
T6	R4	C + 1	CRITICAL	LOAD:76:CRITICAL:STOP	Pass

Save checkpoint_2_boundary_tests.png showing all six outputs and your personalized circuit name. **6.2**

Compare prediction and result

Question	Your answer
Did every observed result match your prediction? If not, what was different?	Yes. Every observed result matches with the expected results. This happened because the reportLoad() checks the CRITICAL_THRESHOLD before WARNING_THRESHOLD.
Why was the automatic for loop not enough to verify W and C?	With STEP = 10 the setup only visits multiples of 10 so it skips 40 and 75 entirely.

7. Practice E — Find and fix a logic bug

This version compiles, but CRITICAL values are reported incorrectly. Run it with $C - 1$, C , and $C + 1$ before changing anything.

```
// BUGGY VERSION

void reportLoad(int load) {
    Serial.print("LOAD:");
    Serial.print(load);
    if (load >= WARNING_THRESHOLD) {
        Serial.println(":WARNING:CHECK");
    } else if (load >= CRITICAL_THRESHOLD) {
        Serial.println(":CRITICAL:STOP");
    } else {
        Serial.println(":NORMAL:CONTINUE");
    }
}

// FIXED VERSION

void reportLoad(int load) {
    Serial.print("LOAD:");
    Serial.print(load);
    if (load >= CRITICAL_THRESHOLD) {
        Serial.println(":CRITICAL:STOP");
    } else if (load >= WARNING_THRESHOLD) {
        Serial.println(":WARNING:CHECK");
    } else {
        Serial.println(":NORMAL:CONTINUE");
    }
}
```

7.1 Debugging steps

Step	What to do
1	Reproduce the problem using $C - 1$, C , and $C + 1$.
2	Write down the actual output without guessing the cause.
3	Compare the output with R3 and R4.
4	Explain which condition accepts the value too early.
5	Make the smallest correction.
6	Run T4–T6 again and save checkpoint_3_bug_fixed.png.

7.2 Bug record

Field	Your record
-------	-------------

Expected behavior	load $\geq$ 75 $\rightarrow$ CRITICAL:STOP
Actual behavior	load $\geq$ 75 still reports WARNING:CHECK
Failed requirement / test	R4 (and T5, T6)
Cause of the bug	WARNING_THRESHOLD is checked first, and since it's smaller than CRITICAL_THRESHOLD, every load that's actually critical also satisfies $\geq$ WARNING_THRESHOLD. It gets caught in the first branch and the CRITICAL check is never reached.
Smallest correction	Swap the order. Swap check $\geq$ CRITICAL_THRESHOLD and $\geq$ WARNING_THRESHOLD
Result after re-testing	T4–T6 pass.

8. Practice F — Improve the structure

After all tests pass, move the six boundary-test calls into a new function named `runBoundaryTests()`. Call it once from `setup()`. Do not change the required output.

8.1 Refactoring check

Question	Your answer
What job does <code>runBoundaryTests()</code> have?	groups all six boundary calls under one clear name.
What became easier to read in <code>setup()</code> ?	It's now a short, readable sequence. Identity print, then one function call, instead of six raw lines mixed in.
Did the output change? What evidence supports your answer?	No. Same six calls, same order, just moved into a function. The printed output is identical.

8.2 Optional extension — choose one

- ☐ Add a fourth status level and define its boundaries clearly.
- ☐ Count how many loop values fall into each status.
- ☐ Change STEP and explain why the boundary tests are still needed.
- ☐ Create a function that prints the student identity.

9. Short reflection

Answer in your own words. Use examples from your code or test results.

Why is a program that compiles not necessarily correct?

Compiling only check if the syntax is a valid C++ program. It says nothing about the logic. If it works or not, or if it matched with the requirements. The bug in part E managed to run cleanly, yet it reported WARNING instead of CRITICAL for loads above 75. This condition is wrong.

What did testing $W - 1$, W , $W + 1$, $C - 1$, C , and $C + 1$ reveal?

It revealed exactly where each branch starts and ends. Testing around W confirms to us that the NORMAL/WARNING boundary, and the C exposed a bug. C and $C+1$ were reported as WARNING instead of CRITICAL.

How did the order of the conditions affect the result?

Because WARNING is smaller than CRITICAL in threshold, checking the WARNING condition first will mean that every CRITICAL load would also satisfy the rule and will get claimed by the WARNING. This means CRITICAL condition would be unreachable.

Where might similar decision rules appear in a web or mobile application?

Similar threshold rules are actually visible in account-balance warnings (normal/low-balance/overdraft). Similarly, are file size checks (fine/nearlimit/rejected). There are also battery indicators (normal, low battery, critical).

10. Submission requirements

SUBMISSION WINDOW Complete the core activity during the Week 2A tutorial. If needed, finish the report and evidence before the deadline announced in BINUSMAYA.

10.1 Submit one ZIP file

<StudentID>_COMP6100001_T02_<Name>.zip Your

ZIP must contain:

- ☐ One completed PDF report.
- ☐ Your final .ino or .txt source file.
- ☐ checkpoint_1_working.png.
- ☐ checkpoint_2_boundary_tests.png.
- ☐ checkpoint_3_bug_fixed.png.
- ☐ Your AI assistance declaration, including "No AI used" when applicable.

10.2 Evidence rules

- Screenshots must be readable and show the relevant code or output.
- At least one screenshot must show the circuit name with your ID and name.
- The submitted source file must match the final verified version.
- A Tinkercad link alone is not enough evidence.
- Do not reuse another student's code, screenshots, results, or reflection.

TINKERCARD LINK:

https://www.tinkercad.com/things/48qwa9oj5nw-t022802499711michael?sharecode=a_GrrJtCY5diqxmULOScuG0KV57G5UM5KeJAifHDfU0

11. Collaboration and AI assistance

You may discuss the meaning of variables, conditions, loops, functions, and error messages. Your final code, evidence, answers, and reflection must be your own and must use your assigned values.

Allowed	Not allowed
Discussing concepts and asking troubleshooting questions.	Sharing a completed report, source file, or screenshots.
Using documentation, TA guidance, or declared AI assistance.	Submitting code or explanations that you cannot explain.
Checking a peer's reasoning by asking questions.	Changing names or tokens to hide copied work.

11.1 AI assistance declaration

The course outline identifies flexible AI use for laboratory work. You remain responsible for correctness, validation, integration, and explanation. Complete the declaration below.

Tool / source	Purpose	Specific assistance used	How I verified it
Claude	To understand the tutorial's requirements	I asked Claude to explain some parts of the codes/requirements for me to submit the files.	I understood the assignment.

☐ I can explain every submitted requirement, code statement, test result, and correction without relying on an external tool.

12. Evaluation rubric

Criterion	Weight	What good work looks like
Personalization and prediction	15%	Correct personal values and completed predictions before execution.
Programming logic	30%	Correct constants, conditions, loop, and reportLoad() function.
Testing	25%	All boundary tests include expected, observed, and pass/fail results.
Debugging and improvement	20%	Bug is reproduced, explained, fixed with a small change, and re-tested; refactoring keeps behavior correct.
Communication and evidence	10%	Answers are clear; files are complete, readable, correctly named, and original.

DEMONSTRATION CHECK The lecturer or TA may ask you to explain or reproduce any part of the submission. Inability to explain personalized code or evidence may trigger further originality review.

13. Final checklist

- ☐ My name, ID, class, circuit name, and identity token are correct.
- ☐ W, C, and STEP match my assigned group.
- ☐ I completed the prediction before recording the output.
- ☐ My program uses if / else if / else, a for loop, and reportLoad(int load).
- ☐ T1–T6 contain observed results and pass/fail decisions.
- ☐ I reproduced and fixed the seeded bug.
- ☐ My refactored version still passes the tests.
- ☐ My three screenshots are readable and correctly named.
- ☐ My final source matches the code shown in my evidence.
- ☐ I completed the AI assistance declaration and checked the ZIP filename.

14. Contact and support

When asking for help, include your class, student ID, name, the section you reached, the exact error or output, and what you already tried. A readable screenshot is helpful.

Channel	Use	Details
BINUSMAYA	Announcements, deadline, and submission	Check this first
Email — preferred	Academic questions and technical problems	ruixin.huang@binus.ac.id
WhatsApp	Urgent or time-sensitive questions	+60 17 744 9523

15. References and further reading

- Arduino. Language Reference. <https://docs.arduino.cc/language-reference/>
- Arduino. Control Structure. <https://docs.arduino.cc/language-reference/en/structure/controlstructure/>
- Arduino. Sketches. <https://docs.arduino.cc/learn/programming/sketches/>
- Kung, D. C. (2023). Software Engineering: An Agile Unified Methodology (2nd ed.). McGraw Hill.
- Sommerville, I. (2015). Software Engineering (10th ed.). Pearson.

BEFORE YOU LEAVE Name one decision rule in an app you use. What are its categories, boundaries, and expected actions?

App: Grab/Gojek - Estimating trip fee/surge

Decision rule: Demand-to-driver ratio in an area determines the surge multiplier shown to riders.

Categories: Normal (1.0x), Moderate surge (1.5x), High surge (2.0x+)

Boundaries: Something like this:

- ratio < 1.2 → Normal
- $1.2 \leq \text{ratio} < 2.0$ → Moderate
- ratio ≥ 2.0 → High

Expected actions:

- Normal → show standard fee and book instantly
- Moderate → show a surge warning before confirming
- High → require explicit rider confirmation of the higher price, sometimes with a "wait for lower price" suggestion (skippable if the rider agrees with the high price)